

# JOGO DA VELHA

GRUPO E: JOÃO HENRIQUE BERGALLO, LARISSA OLIVEIRA DA SILVA E  
MARIA EDUARDA SCHÜLER

# INTRODUÇÃO

**O objetivo deste trabalho foi desenvolver uma solução de Inteligência Artificial capaz de classificar estados de um tabuleiro do jogo da velha em cinco categorias distintas:**

- 0 – Tem Jogo**
- 1 – Possível Fim de Jogo (ameaça)**
- 2 – Empate (tabuleiro cheio)**
- 3 – O vence**
- 4 – X vence**

**Além da construção e avaliação de modelos de Machine Learning, também foi desenvolvido um Front End interativo onde um jogador humano pode jogar contra a máquina, enquanto a IA classifica continuamente o estado do tabuleiro.**

**O trabalho envolveu:**

- **Análise e adaptação de dataset**
- **Engenharia de atributos**
- **Treinamento de múltiplos algoritmos**
- **Ajuste de hiperparâmetros**
- **Comparação entre modelos**
- **Integração da IA com uma aplicação interativa**

**Algoritmos utilizados:**

- **Árvore de Decisão**
- **Random Forest**
- **k-NN (k-Nearest Neighbors)**
- **MLP (Rede Neural Multicamadas)**
- **Agrupamento Hierárquico**

# DATASET E MODIFICAÇÕES REALIZADAS

# DATASET ORIGINAL E PROBLEMAS ENCONTRADOS

O dataset utilizado como base foi o “Tic-Tac-Toe Endgame”, disponibilizado pela UCI Machine Learning Repository disponibilidade no enunciado do trabalho.

O dataset original possui estados finais do jogo da velha e uma classificação binária indicando vitória de X ou não vitória.

Entretanto, o problema proposto exigia um sistema muito mais completo, capaz de reconhecer também:

- estados intermediários;
- possibilidades de fim de jogo;
- empates;
- vitória de O;
- jogos ainda em andamento.

Por esse motivo, o dataset original não atendia completamente aos requisitos do trabalho.

| Problemas                      | Impacto                                         |
|--------------------------------|-------------------------------------------------|
| Apenas estados finais          | Não permitia detectar jogo em andamento         |
| Classificação binária          | Insuficiente para o problema                    |
| Ausência de ameaças de vitória | Não permitia identificar “possibilidade de fim” |
| Pouca diversidade de estados   | Reduziria a generalização dos modelos           |

# NOVO DATASET

**Foi desenvolvido um gerador automático de partidas aleatórias.**

**O sistema simulava milhares de jogos de forma aleatória e armazenava estados únicos do tabuleiro.**

## **Lógica utilizada:**

- 1. iniciar tabuleiro vazio;**
- 2. alternar entre X e O;**
- 3. realizar jogadas aleatórias;**
- 4. classificar o estado atual;**
- 5. armazenar o tabuleiro;**
- 6. encerrar caso exista vitória ou empate.**

**Foi utilizado um conjunto (set) para evitar duplicatas.**

**Cada tabuleiro foi convertido em 16 atributos numéricos.**

**As 9 casas foram codificadas como:**

| Valor | Representação |
|-------|---------------|
| 0     | vazio         |
| 1     | O             |
| 2     | X             |

## **Classe**

## **Significado**

|   |                              |
|---|------------------------------|
| 0 | Tem jogo                     |
| 1 | Possibilidade de fim de jogo |
| 2 | Empate                       |
| 3 | O vence                      |
| 4 | X vence                      |

**A classificação foi feita usando regras determinísticas.**

## **Prioridade utilizada foi:**

- 1. vitória de X;**
- 2. vitória de O;**
- 3. empate;**
- 4. ameaça de vitória;**
- 5. jogo em andamento.**

**A classe “Possibilidade de fim de jogo” foi atribuída quando um jogador possuía:**

- duas peças alinhadas;**
- um espaço vazio restante.**

**Isso representa ameaça imediata de vitória.**

# BALANCEAMENTO E AMOSTRAS POR CLASSE

Durante a geração automática dos estados do jogo, foram produzidas as seguintes quantidades de instâncias únicas:

| Classe                       | Quantidade |
|------------------------------|------------|
| Tem jogo                     | 677        |
| Possibilidade de fim de jogo | 3842       |
| Empate                       | 16         |
| O vence                      | 316        |
| X vence                      | 626        |

Observa-se que a classe “**Possibilidade de Fim de Jogo**” apresentou quantidade muito superior às demais, enquanto a classe “**Empate**” apresentou poucas ocorrências naturais durante as simulações aleatórias. Para reduzir o desbalanceamento, foi aplicada uma estratégia de amostragem, limitando as classes muito grandes a no máximo 1500 exemplos.

Após o balanceamento, o dataset final ficou com 3135 instâncias distribuídas da seguinte forma:

| Classe                       | Quantidade |
|------------------------------|------------|
| Tem jogo                     | 677        |
| Possibilidade de fim de jogo | 1500       |
| Empate                       | 16         |
| O vence                      | 316        |
| X vence                      | 626        |

Mesmo após o balanceamento, a classe “**Empate**” permaneceu pequena devido à baixa frequência natural desse tipo de estado nas simulações aleatórias do jogo.

# DESENVOLVIMENTO DAS SOLUÇÕES

O conjunto de dados foi dividido em:

| Conjunto  | Percentual |
|-----------|------------|
| Treino    | 60%        |
| Validação | 20%        |
| Teste     | 20%        |

Foi utilizado *stratify* para manter a distribuição das classes em todos os conjuntos.

# ÁRVORE DE DECISÃO

Árvore de decisão é um algoritmo supervisionado que utiliza um grafo de regras lógicas (se-então) para tarefas de classificação ou regressão

| Parâmetro         | Valores          |
|-------------------|------------------|
| criterion         | gini, entropy    |
| max_depth         | 4,6,8,10,12,None |
| min_samples_split | 2,5,10           |
| min_samples_leaf  | 1,2,5            |

Também foi aplicada penalização para overfitting.

## Justificativa

Árvores profundas podem decorar os dados. Por isso, o sistema penaliza modelos que apresentam diferença excessiva entre treino e validação.

# RANDOM FOREST

Random Forest é um algoritmo supervisionado de alta precisão e flexibilidade, utilizado tanto para classificação quanto para regressão. A classificação final ocorre por votação.

| Parâmetro    | Valores      |
|--------------|--------------|
| n_estimators | 50,100,150   |
| max_depth    | 5,10,15,None |

## Justificativa

O Random Forest reduz:

- overfitting
- sensibilidade a ruído
- dependência de uma única árvore

Foi utilizado `class_weight='balanced'` para melhorar o tratamento das classes.



# K-NN

O k-NN é um algoritmo "preguiçoso" que classifica novos dados com base na classe mais frequente entre seus k vizinhos mais próximos.

| Parâmetro | Valores           |
|-----------|-------------------|
| k         | 3,5,7,9,11        |
| weights   | uniform, distance |
| p         | 1,2               |

## Justificativa

- valores ímpares evitam empates;
- distância Manhattan (p=1) e Euclidiana (p=2) foram comparadas;
- distance permite dar mais peso aos vizinhos próximos

# MLP

O MLP é uma rede neural com camadas ocultas que resolve problemas não lineares através de propagação de sinais e retropropagação de erro.

Foi utilizado:

- solver Adam
- early stopping
- validação automática

## Topologias testadas

(64)  
(128)  
(64,32)  
(128,64)  
(64,64,32)

## Justificativa

A busca por diferentes arquiteturas permitiu encontrar equilíbrio entre:

- capacidade de aprendizado
- custo computacional
- Generalização

O early stopping foi utilizado para evitar overfitting.

# AGRUPAMENTO HIERÁRQUICO

O agrupamento hierárquico é um algoritmo não supervisionado que organiza dados em grupos aninhados (dendogramas) por similaridade, de forma aglomerativa ou por divisão.

Os dados foram agrupados em 5 clusters.

Depois:

- cada cluster recebeu a classe majoritária
- centroides foram usados para classificação

## Métodos linkages testados

ward

average

complete

## Justificativa

O algoritmo foi utilizado para comparar abordagens supervisionadas e não supervisionadas.

# COMPARAÇÕES E RESULTADOS

**Todos os algoritmos foram avaliados utilizando:**

- **Acurácia**
- **Precision**
- **Recall**
- **F1-Score**

**As métricas foram calculadas no conjunto de teste, utilizando dados não vistos durante o treinamento.**

| Algoritmo               | Acurácia | Precision | Recall | F1-score |
|-------------------------|----------|-----------|--------|----------|
| Árvore de Decisão       | 0.9777   | 0.9778    | 0.9777 | 0.9777   |
| Random Forest           | 0.9777   | 0.9795    | 0.9777 | 0.9772   |
| k-NN                    | 0.9362   | 0.9385    | 0.9362 | 0.9361   |
| MLP                     | 0.9777   | 0.9801    | 0.9777 | 0.9782   |
| Agrupamento Hierárquico | 0.5726   | 0.5740    | 0.5726 | 0.5704   |

| Algoritmo         | Melhores parâmetros                                                           | Acurácia Validação |
|-------------------|-------------------------------------------------------------------------------|--------------------|
| Árvore de Decisão | criterion=gini, max_depth=None,<br>min_samples_split=2,<br>min_samples_leaf=1 | —                  |
| Random Forest     | n_estimators=100,<br>max_depth=15                                             | 0.9761             |
| k-NN              | n_neighbors=7, weights=uniform,<br>p=1                                        | 0.9155             |
| MLP               | hidden_layer_sizes=(64,64,32),<br>learning_rate_init=0.01,<br>momentum=0.5    | 0.9841             |
| Hierárquico       | linkage=ward                                                                  | 0.6093             |

# ÁRVORE DE DECISÃO

A Árvore de Decisão apresentou excelente desempenho, alcançando aproximadamente **97,7% de acurácia** no conjunto de teste.

## Pontos positivos

- alta interpretabilidade
- rápida execução
- facilidade para compreender as regras de decisão

## Limitações

- tendência ao overfitting quando árvores muito profundas são utilizadas

Mesmo utilizando profundidade ilimitada (*max\_depth=None*), o modelo conseguiu manter boa generalização devido ao balanceamento do dataset e ao controle realizado durante a validação.

# RANDOM FOREST

O Random Forest apresentou um dos melhores desempenhos gerais do trabalho.

## Resultados

- Acurácia: 97,77%
- Precision: 97,95%
- Recall: 97,77%
- F1-score: 97,72%

## Pontos positivos

- excelente generalização
- maior robustez
- redução do overfitting
- estabilidade nas previsões

O uso de múltiplas árvores permitiu reduzir a variância do modelo, tornando-o **mais confiável** que uma única árvore de decisão.

## O melhor modelo utilizou:

- 100 árvores
- profundidade máxima igual a 15

# K-NN

O k-NN apresentou desempenho inferior aos modelos baseados em árvores e redes neurais, porém ainda obteve bons resultados.

## Resultados

- Acurácia: 93,62%
- Precision: 93,85%
- Recall: 93,62%
- F1-score: 93,61%

## Melhores parâmetros

- $k = 7$
- distância Manhattan ( $p=1$ )
- pesos uniformes

O algoritmo conseguiu reconhecer padrões do jogo de forma satisfatória, porém apresentou maior **sensibilidade à distribuição espacial das features**.

Como o k-NN depende diretamente da distância entre exemplos, estados semelhantes do tabuleiro podem gerar ambiguidades em algumas situações.

# MLP

A MLP apresentou o melhor desempenho geral entre todos os modelos testados.

## Resultados

- Acurácia: 97,77%
- Precision: 98,01%
- Recall: 97,77%
- F1-score: 97,82%

Melhor topologia encontrada  
Entrada  $\rightarrow (64, 64, 32) \rightarrow$  saída

## Hiperparâmetros

- learning\_rate\_init = 0.01
- momentum = 0.5

Além disso, o relatório de classificação mostrou excelente desempenho em praticamente todas as classes.

A única classe com desempenho ligeiramente inferior foi “O vence”, devido à menor quantidade de exemplos disponíveis no dataset.

A rede neural apresentou excelente capacidade de generalização e conseguiu identificar padrões complexos entre os estados do tabuleiro.

## O uso de:

- múltiplas camadas ocultas
- early stopping
- validação separada ajudou a **evitar overfitting**.

# AGRUPAMENTO HIERÁRQUICO

O Agrupamento Hierárquico apresentou o **pior desempenho** entre os modelos avaliados.

## Resultados

- **Acurácia: 57,26%**
- **Precision: 57,40%**
- **Recall: 57,26%**
- **F1-score: 57,04%**

## Melhor parâmetro

- **linkage = ward**

O desempenho inferior **já era esperado**, pois o algoritmo utilizado é originalmente não supervisionado.

Embora tenha sido possível associar clusters às classes majoritárias, o método apresentou dificuldade em separar corretamente os diferentes estados do jogo.

Mesmo assim, sua utilização foi importante para comparação entre

- **métodos supervisionados**
- **métodos não supervisionados**

# COMPARAÇÃO

| Modelo                  | Destaque                 |
|-------------------------|--------------------------|
| MLP                     | melhor F1-score geral    |
| Random Forest           | maior robustez           |
| Árvore de Decisão       | maior interpretabilidade |
| k-NN                    | simplicidade             |
| Agrupamento Hierárquico | comparação experimental  |

**O algoritmo escolhido foi o MLP. Justificativa:**

- **Obteve o maior F-measure (0.9782) e a maior Precision (0.9801) entre todos os algoritmos — as duas métricas mais relevantes para minimizar falsos positivos de 'fim de jogo'.**
- **Acurácia de validação de 98,41% — a mais alta na fase de seleção de hiperparâmetros.**
- **100% de acerto nas classes Empate, Tem Jogo e Possibilidade de Fim de Jogo no conjunto de teste.**
- **A topologia (64, 64, 32) com Adam e early stopping apresentou excelente generalização sem sinais de overfitting.**
- **Apesar de Árvore de Decisão e Random Forest terem empatado em acurácia com o MLP (0.9777), o MLP superou ambos em F-measure e Precision — métricas que penalizam mais os erros nas classes minoritárias (O vence, Empate), críticas para o correto encerramento do jogo.**



# FRONTEND

Implementação feita com uma aplicação web Flask (*frontend.py*). A interface renderiza o tabuleiro 3x3, permite a interação do jogador humano (X) e simula as jogadas do computador (O) de forma aleatória de acordo com as casas livres.

## Fluxo de uma partida:

1. O jogador clica em uma casa livre – servidor registra a jogada de X
2. Modelo de IA classifica o estado atual do tabuleiro e retorna uma das classes
3. Se a IA detecta X vence, O vence ou Empate – o jogo é encerrado
4. Se a IA detecta que Tem Jogo ou Possibilidade de Fim de Jogo – computador faz uma jogada aleatória em uma casa livre.
5. Após a jogada do computador, a IA classifica novamente. Se detectar O vence ou Empate – encerra, senão - aguarda a próxima jogada do jogador.

A cada classificação realizada durante o jogo, o frontend compara o resultado da IA com o estado real do tabuleiro (verificando as regras determinísticas) e registra:

- Acertos: classificações corretas (IA e regra concordam)
- Erros: classificações incorretas (IA e regra divergem)

A acurácia em tempo real é exibida na interface como: **Acurácia = Acertos / (Acertos + Erros)**. Estes dados são acumulados ao longo de múltiplas partidas.

CONCLUSÃO

# DIFICULDADES

## 1. Inadequação do dataset original:

O dataset do UCI Machine Learning Repository não atendia ao escopo do problema. Ele cobre apenas estados finais onde o X venceu ou não venceu. A solução foi construir um dataset sintético por simulação de partidas aleatórias.

## 2. Raridade de certas classes:

Devido a uma característica matemática do Jogo da Velha: partidas completamente aleatórias, empates genuínos ocorrem em apenas cerca de 13% das partidas. Isso impossibilitou atingir as 200 classes sugeridas no enunciado.

## 3. Desbalanceamento severo e impacto nos modelos:

O desbalanceamento severo apresentou um desafio técnico. Algoritmos como Árvore de Decisão e Random Forest tendem a favorecer classes majoritárias se não houver correção. Foi adotada a solução de ponderamento inverso, penalizando mais os erros nas classes minoritárias.

## 4. Adaptação do Agrupamento Hierárquico:

Adaptar o algoritmo não-supervisionado para classificação supervisionada exigiu um pipeline customizado. Este pipeline apresentou dois problemas, os cluster gerados pelo algoritmo podem não necessariamente corresponderem as classes do problema e a predição por centroide é uma aproximação grosseira que perde informação sobre a real forma de cada cluster.

## 5. Hiperparâmetros do MLP:

Pelo espaço de hiperparâmetros ser vasto, isso tornou o treinamento do MLP significativamente mais lento que os demais algoritmos. Além disso, o MLP é sensível à inicialização aleatória dos pesos garantiu a reprodutibilidade, mas significa que outras sementes poderiam produzir resultados ligeiramente diferentes.

# GANHOS

## **1. Compreensão prática do ciclo de um projeto de Machine Learning:**

- **Análise crítica de um dataset publico**
- **Construção de um dataset sintético adequado**
- **Avaliação com métricas padronizadas**

## **2. Experiência com desbalanceamento de classes, um problema clássico e frequente.**

## **3. Comparação empírica entre cinco paradigmas distintos de aprendizagem, permitindo entender na prática quando cada abordagem se sai melhor e por quê.**

## **4. Contato com o scikit-learn em profundidade, não apenas chamando fit/predict, mas entendendo parâmetros como class\_weight, stratify, early\_stopping e suas implicações nos resultados.**

## **5. Desenvolvimento da capacidade de interpretar e justificar resultados**

# IA COMO FERRAMENTA DE AUXÍLIO

**Durante o desenvolvimento deste trabalho foi utilizado Claude, NotebookLM e Copilot como instrumentos de apoio em etapas específicas do trabalho. O uso foi sempre complementar ao esforço humano, sendo as decisões técnicas, lógica do problema e análise dos resultados conduzidas pelo próprio grupo.**

**Auxílio na redação do relatório/apresentação:**

**O Copilot e NotebookLM foram utilizados na redação e organização desta apresentação e do relatório.**

**Contribuindo para os seguintes aspectos:**

- **Estruturação: sugestão de seções, subseções e ordem lógica de apresentação dos conteúdos**
- **Revisão e refinamento textual: melhoria da clareza, coesão e precisão técnica dos parágrafos**
- **Padronização da linguagem: manutenção de terminologia consistente ao longo do documento**

**Todos os dados, métricas, análises e conclusões presentes são baseadas nos resultados reais obtidos pela execução dos scripts python desenvolvidos pelo grupo.**

**Auxílio no desenvolvimento estético do frontend:**

**O Claude.ai foi utilizado como ferramenta de apoio no desenvolvimento da interface visual do Jogo da Velha. O backend lógico do jogo foi desenvolvido pelo próprio grupo.**

- **Layout e estrutura HTML/CSS**
- **Paleta de cores e tipografia**
- **Animações e transições CSS**
- **Mensagens ao usuário**

**O uso da IA nesta etapa permitiu que o grupo se dedicasse mais tempo à parte técnica do trabalho sem comprometer a qualidade visual da entrega.**

# CONSIDERAÇÕES FINAIS

**O uso das ferramentas de IA foi transparente e responsável. Todo o conteúdo gerado com auxílio de IA foi revisado, validado e adaptado pelo grupo antes de ser incorporado no trabalho. Não foram usados algoritmos para fabricar ou alterar resultados experimentais. Fazendo com que o trabalho atenda aos requisitos acadêmicos de autoria e originalidade.**

**Este projeto demonstrou o desenvolvimento completo de um classificador de estados do Jogo da Velha utilizando cinco algoritmos de aprendizado de máquina. O dataset sintético gerado por simulação de 80.000 partidas se mostrou mais adequado que o dataset original do UCI, cobrindo as cinco classes necessárias, incluindo estado intermediários do jogo.**

OBRIGADO

# JOGO DA VELHA

GRUPO E: JOÃO HENRIQUE BERGALLO, LARISSA OLIVEIRA DA SILVA E  
MARIA EDUARDA SCHÜLER

DISCIPLINA: INTELIGÊNCIA ARTIFICIAL

PROFESSORA: DR<sup>a</sup> SILVIA MORAES

2026/01